

Graphs Study Guide

Graphs Study Guide

1. Overview

Graphs are one of the most powerful and flexible data structures in computer science. They model relationships between entities and appear in routing, networking, social networks, dependency resolution, and countless LeetCode problems.

A graph consists of:

- **Nodes (vertices)**
- **Edges** (directed or undirected)

Common representations:

- **Adjacency list** (most common)
- **Adjacency matrix** (dense graphs)
- **Edge list**

Graphs are ideal when you need to model:

- Connectivity
- Reachability
- Shortest paths
- Cycles and dependencies

2. Core Graph Traversal Patterns

2.1 [Breadth-First Search \(BFS\)](#)

Used for:

- Shortest path in unweighted graphs
- Level-by-level exploration
- Flood fill / grid traversal

Template:

```
from collections import deque
```

```
def bfs(start):  
    queue = deque([start])  
    visited = {start}  
  
    while queue:  
        node = queue.popleft()  
        for nei in graph[node]:  
            if nei not in visited:  
                visited.add(nei)  
                queue.append(nei)
```

2.2 [Depth-First Search \(DFS\)](#)

Used for:

- Cycle detection
- Connected components
- Topological sorting (DFS version)
- Backtracking on graphs

Template:

```
def dfs(node):  
    if node in visited: return  
    visited.add(node)  
    for nei in graph[node]:  
        dfs(nei)
```

2.3 [Topological Sort](#)

Used for **directed acyclic graphs (DAGs)**.

Applications:

- Course Schedule
- Build order
- Dependency resolution

Two methods:

- DFS postorder
- BFS (Kahn's Algorithm)

3. Shortest Path Algorithms

3.1 [Dijkstra's Algorithm](#)

Used for weighted graphs with **non-negative** edges.

Key idea:

- Use a min-heap to always expand the closest node.
-

3.2 [Bellman-Ford](#)

Used when edges may be **negative**.

Detects negative cycles.

3.3 [Floyd-Warshall](#)

Used for **all-pairs shortest paths**.

Time complexity: $O(n^3)$

3.4 [BFS for Unweighted Graphs](#)

Shortest path = BFS levels.

4. Graph Representations

4.1 Adjacency List

```
graph = {  
  0: [1, 2],  
  1: [2],  
  2: [0, 3],  
  3: [3]  
}
```

4.2 Adjacency Matrix

```
matrix = [  
  [0, 1, 1, 0],  
  [0, 0, 1, 0],  
  [1, 0, 0, 1],  
  [0, 0, 0, 1]  
]
```

5. Example Problem: Number of Islands (Grid BFS)

```
def numIslands(grid):
    if not grid: return 0
    rows, cols = len(grid), len(grid[0])
    visited = set()

    def bfs(r, c):
        queue = [(r, c)]
        visited.add((r, c))
        while queue:
            x, y = queue.pop(0)
            for dx, dy in [(1,0),(-1,0),(0,1),(0,-1)]:
                nx, ny = x+dx, y+dy
                if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == '1' and (nx, ny) not in visited:
                    visited.add((nx, ny))
                    queue.append((nx, ny))

    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1' and (r, c) not in visited:
                bfs(r, c)
                count += 1
    return count
```

6. Common Pitfalls

- Forgetting to mark nodes as visited
- Infinite loops from revisiting neighbors
- Using DFS on deep graphs → recursion depth errors
- Misidentifying directed vs undirected edges
- Not handling disconnected components

7. Practice Roadmap

Beginner

- Number of Islands
- Flood Fill

- Clone Graph

Intermediate

- Course Schedule (Topological Sort)
- Rotting Oranges
- Shortest Path in Binary Matrix

Advanced

- Dijkstra's Algorithm problems
- Bellman-Ford problems
- Minimum Spanning Tree (Kruskal/Prim)
- Word Ladder

8. Key Takeaways

- Graphs model relationships and connectivity.
- BFS and DFS are the foundation of most graph problems.
- Weighted graphs require shortest-path algorithms.
- DAGs enable topological sorting.

End of document.